Frontend Developer Internship

A 6-month, 24-week program: HTML → CSS → JavaScript → React → Capstone

Starter code & templates: https://github.com/mahfoos/frontend-internship-program

Program overview

Interns progress from zero to shipping a production-quality React application over 24 weeks. Each week has concrete tasks and one deliverable, reviewed through a GitHub pull request. Each month ends with a milestone review. Starter code for every month is in the companion repository.

MONTH	WEEKS	FOCUS & OUTCOME
Month 1	Weeks 1–4	**HTML & Web Foundations** A 3-page personal portfolio in semantic HTML with zero W3C validator errors.
Month 2	Weeks 5–8	**CSS & Responsive Design** A responsive landing page (360px–1440px) with light/dark themes, deployed to GitHub Pages.
Month 3	Weeks 9–12	**JavaScript** A persisted to-do app and an API-driven search app in vanilla JavaScript.
Month 4	Weeks 13–16	**React Fundamentals** The to-do and search apps rebuilt in React, plus a small reusable component library.
Month 5	Weeks 17–20	**Advanced React & Professional Practice** A tested multi-page store front with routing, a cart and CI-ready quality checks.
Month 6	Weeks 21–24	**Capstone Project** A deployed capstone app with tests, documentation and a recorded demo.

Weekly rhythm

Monday	Plan the week: read the week's tasks, create issues on your board.
Daily	15-minute stand-up (yesterday / today / blockers) and at least one commit.
Wednesday	Mid-week check-in with your mentor; open a draft PR.

Friday	Demo, merge reviewed PRs, submit the weekly report (templates/weekly-report.md).

Assessment rubric

Each monthly milestone and the final capstone are scored with this rubric (1–5 per area, weighted).

AREA	WEIGHT	WHAT GOOD LOOKS LIKE
Functionality	25%	Features work as specified, edge cases and errors handled.
Code quality	25%	Readable, consistent, small components/functions, no dead code.
Accessibility & responsiveness	15%	Semantic markup, keyboard support, 360–1440px layouts.
Git & collaboration	15%	Small focused PRs, clear commit messages, constructive reviews.
Communication	20%	Weekly reports on time, clear demos, asks for help early.

Month 1 · HTML & Web Foundations

Set up a professional workflow and learn to write valid, semantic, accessible HTML.

> **Outcome:** A 3-page personal portfolio in semantic HTML with zero W3C validator errors.
> **Starter:** `month-1-html/starter`

Week 1 Dev environment, Git & GitHub

- ☐ Install VS Code, Node.js LTS, Git and a modern browser; enable format-on-save.
- ☐ Configure Git (name, email, SSH key) and learn clone, status, add, commit, push, pull.
- ☐ Practise branching: create a branch, open a pull request, respond to review comments.
- ☐ Fork this repository and add yourself to INTERNS.md through a pull request.

Deliverable: Merged PR adding you to INTERNS.md, plus your first weekly report.

Week 2 HTML document structure

- ☐ Learn doctype, <html>, <head>, <meta>, <title> and how the browser parses a page.
- ☐ Use headings, paragraphs, lists, links (relative/absolute) and images with width/height.
- ☐ Complete month-1-html/starter/index.html: a personal home page with a short bio.
- ☐ Use browser DevTools to inspect the DOM of three websites you like.

Deliverable: A home page with correct document structure and at least one image and list.

Week 3 Semantic HTML & accessibility

- ☐ Rebuild layouts with header, nav, main, section, article, aside and footer.
- ☐ Write meaningful alt text, keep a logical heading order, add a skip link.
- ☐ Link the three starter pages (home, about, contact) with a shared nav.
- ☐ Run Lighthouse and reach an accessibility score of 90+.

Deliverable: 3-page site with shared navigation and a Lighthouse accessibility score ≥ 90.

Week 4 Forms, tables & month-1 review

- ☐ Build the contact form: labels, fieldset/legend, input types, required, pattern.
- ☐ Build a weekly timetable using <table>, <caption>, <thead> and scope attributes.
- ☐ Validate every page with the W3C validator and fix all errors.
- ☐ Demo your portfolio to your mentor; collect feedback as GitHub issues.

Deliverable: Milestone 1: validated 3-page portfolio, reviewed and merged.

Resources

- MDN: Getting started with the web — https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web
- MDN: HTML basics — https://developer.mozilla.org/en-US/docs/Learn/HTML
- Pro Git (chapters 1–3) — https://git-scm.com/book/en/v2
- W3C Markup Validator — https://validator.w3.org/

Month 2 · CSS & Responsive Design

Style pages confidently with modern layout, responsive design and maintainable CSS.

> **Outcome:** A responsive landing page (360px–1440px) with light/dark themes, deployed to GitHub Pages.
> **Starter:** `month-2-css/starter`

Week 5 Selectors, cascade & the box model

- ☐ Learn selectors, specificity, inheritance and the cascade order.
- ☐ Master the box model: margin, border, padding, box-sizing.
- ☐ Use rem, em, %, vw/vh and know when each is appropriate.
- ☐ Style your month-1 portfolio with an external stylesheet (no inline styles).

Deliverable: Portfolio styled with a consistent type scale and spacing.

Week 6 Flexbox

- ☐ Finish all 24 levels of Flexbox Froggy.
- ☐ Build a navbar that collapses gracefully on small screens.
- ☐ Build a row of feature cards with equal heights and wrapping.
- ☐ Centre content horizontally and vertically three different ways.

Deliverable: Header and feature-card sections of the month-2 starter, built with Flexbox.

Week 7 Grid & responsive layouts

- ☐ Finish Grid Garden; learn grid-template-areas, fr units, minmax() and auto-fit.
- ☐ Adopt a mobile-first approach with min-width media queries.
- ☐ Use clamp() for fluid typography and responsive images with srcset.
- ☐ Test layouts at 360, 768, 1024 and 1440px in DevTools device mode.

Deliverable: Responsive gallery/pricing section that works at all four breakpoints.

Week 8 Theming, animation & month-2 review

- ☐ Move colours and spacing into CSS custom properties (variables.css).
- ☐ Add a dark theme with prefers-color-scheme; respect prefers-reduced-motion.
- ☐ Add hover/focus states and subtle transitions; name classes with BEM.
- ☐ Deploy the landing page to GitHub Pages.

Deliverable: Milestone 2: deployed responsive landing page with light/dark themes.

Resources

- MDN: CSS first steps — https://developer.mozilla.org/en-US/docs/Learn/CSS/First_steps
- Flexbox Froggy — https://flexboxfroggy.com/
- Grid Garden — https://cssgridgarden.com/
- web.dev: Learn Responsive Design — https://web.dev/learn/design/

Month 3 · JavaScript

Learn the language, manipulate the DOM and talk to real APIs.

> **Outcome:** A persisted to-do app and an API-driven search app in vanilla JavaScript.
> **Starter:** `month-3-javascript/starter`

Week 9 Language fundamentals

- ☐ Variables (let/const), types, operators, equality, truthy/falsy.
- ☐ Conditionals, loops, functions, arrow functions, scope and closures.
- ☐ Implement every function in starter/exercises/week-09.js.
- ☐ Run npm test inside starter/ until all week-9 tests pass.

Deliverable: All week-9 exercises passing.

Week 10 Arrays, objects & modules

- ☐ Array methods: map, filter, reduce, find, some, every, sort.
- ☐ Objects, destructuring, spread/rest, optional chaining.
- ☐ Split code into ES modules with import/export.
- ☐ Implement every function in starter/exercises/week-10.js.

Deliverable: All week-10 exercises passing.

Week 11 DOM & events

- ☐ querySelector, createElement, textContent vs innerHTML (and why XSS matters).
- ☐ Event listeners, event delegation, form submit handling.
- ☐ Complete starter/todo-app: add, toggle, delete, filter (all/active/done).
- ☐ Persist todos in localStorage so they survive a reload.

Deliverable: Working, keyboard-accessible to-do app.

Week 12 Async JavaScript & month-3 review

- ☐ Promises, async/await, try/catch and the event loop.
- ☐ fetch() with loading, empty and error states.
- ☐ Complete starter/api-app: search GitHub users and show their profile cards.
- ☐ Debounce the search input; handle API rate-limit errors gracefully.

Deliverable: Milestone 3: to-do app + GitHub user search, both deployed.

Resources

- javascript.info (Part 1 & 2) — https://javascript.info/
- MDN: JavaScript Guide — https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
- MDN: Using the Fetch API — https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- Node.js test runner — https://nodejs.org/api/test.html

Month 4 · React Fundamentals

Think in components, manage state and fetch data the React way.

> **Outcome:** The to-do and search apps rebuilt in React, plus a small reusable component library.
> **Starter:** `month-4-react/starter`

Week 13 Tooling, JSX, components & props

- ☐ npm basics: package.json, scripts, dependencies vs devDependencies.
- ☐ Run the Vite starter; learn JSX rules and component file structure.
- ☐ Break your portfolio into Header, Hero, ProjectCard and Footer components.
- ☐ Pass data with props; render lists with stable keys.

Deliverable: Portfolio rebuilt as React components.

Week 14 State & events

- ☐ useState, event handlers, controlled inputs.
- ☐ Lifting state up and passing callbacks to children.
- ☐ Conditional rendering and derived state (avoid duplicating state).
- ☐ Port the month-3 to-do app into src/features/todos.

Deliverable: React to-do app with the same features as the vanilla version.

Week 15 Effects, data fetching & custom hooks

- ☐ useEffect, dependency arrays and cleanup; AbortController for fetches.
- ☐ Implement the useFetch hook in src/hooks/useFetch.js.
- ☐ Port the GitHub user search using useFetch and a debounced query.
- ☐ Handle loading, error and empty states as components.

Deliverable: React GitHub search powered by your own useFetch hook.

Week 16 Component design & month-4 review

- ☐ Build reusable Button, Card, Modal and Input components (src/components/ui).
- ☐ Use composition and the children prop instead of deep prop lists.
- ☐ Style with CSS Modules; keep components accessible (focus trap in Modal).
- ☐ Write a short usage doc for each component in the README.

Deliverable: Milestone 4: component library + both apps, reviewed and merged.

Resources

- react.dev: Learn — https://react.dev/learn
- react.dev: Thinking in React — https://react.dev/learn/thinking-in-react
- Vite guide — https://vite.dev/guide/
- react.dev: You Might Not Need an Effect — https://react.dev/learn/you-might-not-need-an-effect

Month 5 · Advanced React & Professional Practice

Routing, shared state, testing, performance and code review — how teams ship React.

> **Outcome:** A tested multi-page store front with routing, a cart and CI-ready quality checks.
> **Starter:** `month-5-react-advanced/starter`

Week 17 Routing

- ☐ Set up React Router: layout routes, nested routes, URL params, 404 page.
- ☐ Build Home, Products, ProductDetail and Cart pages in the starter.
- ☐ Load products from https://fakestoreapi.com/products (or the local mock).
- ☐ Keep search/filter state in the URL with useSearchParams.

Deliverable: Navigable product catalogue with detail pages and deep links.

Week 18 State management with reducer + context

- ☐ Model cart actions (add, remove, change quantity, clear) in cartReducer.
- ☐ Expose the cart through CartContext and a useCart hook.
- ☐ Show item count in the header and totals on the cart page.
- ☐ Persist the cart to localStorage.

Deliverable: Fully working cart shared across pages.

Week 19 Testing

- ☐ Unit-test cartReducer for every action (pure functions first).
- ☐ Component-test pages with React Testing Library — query by role, not class.
- ☐ Mock fetch for product loading; test loading and error states.
- ☐ Reach ≥ 70% coverage on src/context and src/pages (npm run coverage).

Deliverable: Green test suite with ≥ 70% coverage on cart and pages.

Week 20 Performance, quality & month-5 review

- ☐ Profile with React DevTools; apply memo/useMemo/useCallback only where measured.
- ☐ Lazy-load routes with React.lazy and Suspense.
- ☐ Run Lighthouse (performance, a11y, best practices ≥ 90).
- ☐ Review two teammates' pull requests using the review checklist.

Deliverable: Milestone 5: optimised, tested store front + two written peer reviews.

Resources

- React Router docs — https://reactrouter.com/
- react.dev: Scaling up with reducer and context — https://react.dev/learn/scaling-up-with-reducer-and-context
- Testing Library: React — https://testing-library.com/docs/react-testing-library/intro/
- Vitest guide — https://vitest.dev/guide/

Month 6 · Capstone Project

Plan, build, test, deploy and present a production-quality app on your own.

> **Outcome:** A deployed capstone app with tests, documentation and a recorded demo.
> **Starter:** `month-6-capstone/starter`

Week 21 Proposal & planning

- ☐ Pick a project (see ideas below) and fill in docs/PROPOSAL.md.
- ☐ Write user stories with acceptance criteria; sketch wireframes.
- ☐ Create GitHub issues and a project board with milestones for weeks 22–24.
- ☐ Get mentor sign-off on scope before writing feature code.

Deliverable: Approved proposal, wireframes and a populated project board.

Week 22 Build core features

- ☐ Set up routing, layout and the data layer (src/services).
- ☐ Ship the must-have user stories behind small, reviewed PRs.
- ☐ Write tests alongside each feature.
- ☐ Daily stand-up note: yesterday, today, blockers.

Deliverable: All must-have stories working on the main branch.

Week 23 Complete, harden & test

- ☐ Finish should-have stories; form validation and error boundaries.
- ☐ Responsive pass (360–1440px) and accessibility pass (keyboard, screen reader, contrast).
- ☐ npm run lint and npm test clean; CI green on every PR.
- ☐ Fix all bugs filed by mentor/peer testing.

Deliverable: Feature-complete, tested and accessible app.

Week 24 Deploy, document & present

- ☐ Deploy to Vercel, Netlify or GitHub Pages; add the live URL to the README.
- ☐ Write the README: features, screenshots, tech stack, setup, decisions.
- ☐ Present a 10-minute demo and a short retrospective to the team.
- ☐ Final evaluation with your mentor using the rubric.

Deliverable: Final milestone: live app, polished README, demo and evaluation.

Capstone ideas

- Personal finance / expense tracker with charts and budgets
- Recipe finder with filters, favourites and a shopping list
- Job board with search, saved jobs and application tracking
- Habit tracker with streaks and calendar heat-map
- Event booking UI with seat selection and a checkout flow

Resources

- GitHub Projects (planning boards) — https://docs.github.com/en/issues/planning-and-tracking-with-projects
- Vercel: deploy a Vite app — https://vercel.com/docs/frameworks/vite
- Netlify: deploy a site — https://docs.netlify.com/site-deploys/create-deploys/
- WCAG 2.2 quick reference — https://www.w3.org/WAI/WCAG22/quickref/